

Senior AI Agent Engineer 2026 — интенсив с ИИ-тьютором

Программа состоит из 14 модулей, 136 практических промптов и сквозного проекта для промышленной эксплуатации. Полная программа рассчитана на 502 часа; ускоренное прохождение с досрочной модульной защитой — на 350–400 часов.

Язык плана и уроков

Объяснения, переходы, названия обычных архитектурных операций и выводы пишутся по-русски. Английскими остаются только:

- фундаментальные термины ИИ: LLM, Agent, tool, tool call, prompt, context, RAG;
- точные названия протоколов, продуктов и стандартов: MCP, A2A, AG-UI, OpenTelemetry, OpenAI Responses и другие;
- имена классов, полей, команд и фрагменты кода.

При первом появлении термин объясняется простыми словами и на конкретном примере. Нельзя подменять объяснение цепочкой англоязычных слов. Если у понятия есть ясный русский эквивалент — «состояние», «повторная попытка», «крайний срок», «побочный эффект», «поток выполнения» — используется он.

Фундаментальный термин	Что он означает в этой программе
LLM	Большая языковая модель, которую приложение вызывает через API
Agent	Система, где LLM предлагает следующий шаг, а код управляет состоянием, доступными tool и завершением
tool	Разрешённая функция или внешний API, который может вызвать система
tool call	Структурированное предложение LLM вызвать tool; до проверки это не команда
prompt	Инструкция и входные данные, переданные LLM
context	Все данные, которые LLM видит в конкретном вызове
RAG	Получение внешних данных перед ответом LLM с опорой на найденные источники

В модулях про LLM слово `Model` означает вызываемую через API LLM, например GPT, Claude или Gemini. Более широкое имя применяется только в точных контрактах вроде `ModelClient` и `ModelResponse`.

Формат обучения: чтение кода, созданного ИИ

Код пишет ИИ-тьютор. Студент не начинает с пустого файла и не тратит время на ручной шаблонный код. Его работа ближе к современной роли ведущего инженера:

1. Понять теорию и задать вопросы до реализации.
2. Согласовать границы, упрощения и ожидаемое поведение.
3. Прочитать сгенерированный код и проследить один запрос от входа до ответа.
4. Объяснить, зачем нужен каждый существенный компонент.
5. Предсказать поведение при изменении входа или отказе зависимости.
6. Проверить изменения, тесты, трассу выполнения и метрики, а не доверять ответу LLM.

7. Попросить изменить реализацию и оценить последствия изменения.
8. Защитить архитектурное решение и удалить неоправданную сложность.

Способность напечатать код по памяти не является критерием прохождения. Модульная защита подтверждает, что студент понимает систему, способен направлять Agent для написания кода и отличает рабочую реализацию от правдоподобной, но неверной.

Как пользоваться ИИ-тьютором

1. Используй этот репозиторий как единое учебное пространство.
2. В начале новой сессии отправляй мастер-промт.
3. Запускай тему командой `начать <номер промпта>`.
4. Сначала тьютор создаёт конспект Markdown/PDF и даёт короткое объяснение.
5. Задавай вопросы; когда теория понятна, напиши `к практике`.
6. Тьютор сам реализует код и тесты, затем проводит пошаговый разбор.
7. Используй команды `покажи поток запроса`, `разбери следующий файл`, `сломай пример`, `покажи diff` и `проверь моё объяснение`.
8. После темы тьютор обновляет конспект и PDF.
9. Не сопоставляй решение с фреймворком до проверки собственной реализации механизма.
10. В каждом модуле сохраняй ADR, отчёт об оценке и git tag.

Ускоренное прохождение

- После короткой теории попроси показать карту архитектуры и сразу переходи к реализации, созданной Agent, если тема уже знакома.
- Если артефакт проходит критерии приёмки и тесты отказов, сокращай разбор до одной сквозной трассы, проверки изменений и отличий от фреймворка.
- Не пропускай модули 2, 6, 7, 12, 13 и 14; в них можно сократить чтение, но не реализацию, проверку кода и доказательства работоспособности.
- В модуле 8 поручай тьютору создавать минимальную версию каждого паттерна, а глубоко проверяй только три, выбранные измеримым сравнением.
- Экономия времени засчитывается только по исполняемым доказательствам, а не по самооценке «я это уже знаю».

Мастер-промт

Ты — главный ИИ-инженер, мой технический тьютор и Agent для написания кода. Я старший Python-разработчик с DevOps/SRE-опытом: не объясняй базовый Python, Git, Docker, Kubernetes, SQL, OAuth и CI/CD, но устройство системы Agent объясняй с первого принципа и на конкретных примерах.

Пиши объяснения по-русски. Английскими оставляй только фундаментальные термины ИИ (LLM, Agent, tool, tool call, prompt, context, RAG), точные названия протоколов и продуктов, имена классов, полей, команд и кода. При первом употреблении сразу объясняй смысл термина.

Каждую тему води в двух фазах. Фаза «Теория»: создай или обнови конспект в `notes/module_XX_topic/` в Markdown и PDF. Объясни проблему, ключевые понятия, один простой поток выполнения, ограничения и типичные ошибки. В чате дай компактное объяснение без огромных листингов и остановись для моих вопросов. Не переходи к коду, пока я не напишу `к практике`.

Фаза «Практика»: сначала покажи план и дерево файлов, затем самостоятельно реализуй рабочий типизированный код на Python и детерминированные тесты без фреймворков для Agent. Я не пишу реализацию с пустого файла. После проверок проведи пошаговый разбор кода: покажи одну сквозную трассу, затем разбирай не

больше одного-двух компонентов за ответ. Не вставляй полный код в чат — ссылайся на файлы и цитируй только ключевые фрагменты.

Проверяй моё понимание через вопросы на предсказание поведения, чтение изменений, поиск намеренно добавленной ошибки и объяснение результатов тестов. Если я ошибся, вернись к конкретному участку кода и эксперименту. После разбора обнови конспект и PDF.

В реализации используй асинхронный типизированный Python, детерминированные подмены, модульные, контрактные, интеграционные тесты и тесты свойств. Добавляй крайние сроки, отмену, лимиты и телеметрию только там, где они действительно нужны. Ищи опасные повторные попытки, дублирование побочных эффектов, гонки, внедрение вредоносных инструкций, расхождение схем, утечки между арендаторами и лишнюю сложность из-за Agent.

Для изменчивых API используй первичные источники, указывай `verified_at`, проверенные версии спецификации и SDK, а также известные устаревания. Сопоставление с фреймворком показывай только после собственной реализации. Если Agent не даёт измеримого выигрыша относительно простого базового решения, рекомендуй убрать его.

Ожидаемая структура ответа тьютора

1. Путь к созданному конспекту Markdown/PDF.
2. Короткая карта темы и простой поток выполнения.
3. Пауза для вопросов.
4. После `к практике`: план, дерево файлов и критерии приёмки.
5. Реализация тьютором и результаты тестов.
6. Пошаговый разбор кода и одна сквозная трасса.
7. Эксперименты с отказами и безопасностью, набор данных, базовое решение и метрики.
8. Проверка понимания по коду и изменениям.
9. Сопоставление с фреймворком после модульной защиты.
10. Обновлённый конспект, проверка готовности к эксплуатации и следующий шаг.

Как исполнять предметные промпты

Каждый пронумерованный промпт ниже — спецификация всей темы, а не одного ответа. Даже если в нём сразу описана реализация, тьютор обязан разделить работу:

1. Сначала выполнить только U.1: создать конспект, кратко объяснить тему и остановиться.
2. Ответить на вопросы по теории.
3. Только после команды `к практике` выполнить U.2–U.8 небольшими шагами.
4. Завершить тему через U.9–U.10 и обновить конспект.

Это правило имеет приоритет над формулировками отдельных предметных промптов. Команда «реализуй» всегда адресована тьютору, а не студенту.

Сквозной проект

Все модули развивают `Evidence & Change Agent`, который исследует версии корпуса знаний и работающие системы, строит план, вызывает tool, создаёт отчёт с доказательствами или набор изменений, работает в песочнице, запрашивает точечное подтверждение опасных действий, переживает падение процесса и предоставляет трассу, оценку и журнал аудита.

Карта трека

Модуль	Часы	Артефакт
1. Среда выполнения LLM	18	Шлюз LLM, независимый от поставщика
2. Структурированные ответы и вызовы tool	28	Безопасная среда выполнения tool
3. Проектирование context	24	Компилятор context
4. Поиск и RAG с участием Agent	32	Гибридная база знаний
5. Системы памяти	28	Типизированная гибридная память
6. Собственный фреймворк Agent	64	Среда выполнения Agent v0-v5
7. Надёжное выполнение графа	32	Отказоустойчивый исполнитель графа
8. Расширенные паттерны Agent	44	Измеримое сравнение паттернов
9. Система нескольких Agent	24	Среда координатора и исполнителей
10. Проектирование протоколов	24	Демонстрация MCP/A2A/AG-UI
11. Внутреннее устройство фреймворков	16	Сравнительный разбор и ADR
12. Безопасность Agent высокого риска	32	Песочница и набор атакующих проверок
13. Оценка, наблюдаемость и оптимизация	40	Платформа оценки и выпуска
14. Итоговый промышленный проект	96	Защита готовности к эксплуатации
Итого	502	

Модуль 1: Среда выполнения LLM и абстракция поставщика

В этом модуле LLM — это GPT, Claude, Gemini или другая большая языковая модель, доступная через API. Цель — отделить вероятностный компонент LLM от среды выполнения Agent и создать границу адаптера без потери данных. Это не обзор SDK, а исследование потоковых ответов, частичных отказов, возможностей поставщиков, учёта потребления и стоимости генерации.

Ключевые темы

1. Токенизация, обработка входного prompt, генерация токенов, время до первого токена (TTFT) и пропускная способность.
2. События потокового ответа и частично полученный ответ.
3. Вероятностные стратегии выбора следующего токена, бюджет рассуждения, остановка и отказ.
4. Реестр возможностей и резервный маршрут.
5. Кэш KV, кэш префиксов и кэширование prompt у поставщика.
6. Запись и воспроизведение запросов, детерминированная имитация LLM.



Набор промптов

Промпт 1.1: LLM — ещё не Agent

Начни только с фазы «Теория». На одном примере — обработка запроса пользователя с возможным вызовом tool, то есть разрешённой функции приложения, — объясни разницу между клиентом LLM (`ModelClient`), циклом Agent, заранее заданным процессом и распределённым исполнителем. Покажи одну общую схему и компактную сравнительную таблицу: что компонент знает, за что отвечает и какие ошибки возвращает наверх. Не выдавай интерфейсы и большой листинг. Создай конспект модуля 1 в Markdown/PDF и остановись для вопросов.

После моей команды к практике самостоятельно создай минимальный запускаемый пример без готовых фреймворков Agent: четыре маленьких компонента, одну трассу успешного сценария и несколько тестов граничных случаев. Затем покажи дерево файлов и проведи разбор кода от входного запроса до результата, по одному-двум файлам за ответ.

Промпт 1.2: Независимый от поставщика контракт

Сначала обнови конспект: на трёх коротких примерах ответов поставщиков объясни, зачем нужны `ModelClient`, `ModelRequest`, `ModelResponse` и `ModelEvent`, что можно привести к общей форме, а что необходимо сохранить без потерь. Покажи классификацию ошибок и матрицу контрактных тестов без полного кода.

После к практике сам реализуй общие контракты и детерминированную имитацию LLM. Поддержи текст, структурированный ответ, вызов tool, сведения об использовании, отказ и события потокового ответа, сохранив специфичные для поставщика блоки. Запусти контрактные тесты, затем разбери со мной модели данных и одну трассу успешных и ошибочных событий.

Промпт 1.3: Поточковый ответ и отмена

Кратко объясни жизненный цикл потокового ответа и обнови конспект одной временной диаграммой. После к практике самостоятельно расширь адаптер и создай эксперимент с отказами: поток оборвался, клиент отменил запрос, поставщик вернул 429/Retry-After, ответ завершился из-за лимита токенов, сведения об использовании отсутствуют. Реализуй передачу крайнего срока и отмены, добавь тесты. Затем пошагово покажи две трассы: успешную отмену и неоднозначный частичный ответ. Вместе определим, какие повторы безопасны, а какие меняют семантику.

Промпт 1.4: Возможности и резервный маршрут

Сначала на понятном примере объясни согласование возможностей и обнови конспект. После к практике сам реализуй реестр без `if model_name`. Описание возможностей должно включать доступные tool, строгую схему, модальности, context, потоковый ответ и экспериментальные функции. Добавь маршрутизатор с резервным выбором, который не может молча потерять требуемую возможность, и тесты свойств для несовместимой резервной LLM. Затем покажи трассу решения маршрутизатора и предложи мне предсказать результат для трёх конфигураций.

Промпт 1.5: Устройство кэшей

Объясни кэш KV, кэш префиксов и кэширование prompt у поставщика через предварительное заполнение и генерацию токенов, затем добавь сравнение в конспект. После к практике сам создай и запусти эксперимент: неизменяемый префикс, изменение одного токена, несколько арендаторов и конфиденциальные данные. Измерь долю попаданий в кэш, TTFT и стоимость, покажи таблицу результатов и код измерения. Затем попроси меня объяснить по трассе, почему кэш не является памятью.

Промпт 1.6: Запись, воспроизведение и оценка

Сначала объясни запись и воспроизведение на одной короткой трассе и обнови конспект. После к практике сам реализуй адаптер и детерминированную имитацию LLM для обычного ответа, ответа с нарушенной структурой, вызова tool, частичного потока, превышения времени и отмены. Добавь средство запуска JSONL, сохраняющее события, результат, задержку и сведения об использовании. Запусти тесты и разбери со мной один файл воспроизведения. Заверши сравнением детерминированных контрактных тестов и повторных оценок через настоящий API.

Критерии прохождения модуля

- Два адаптера проходят один набор контрактных тестов.
- Метаданные поставщика не теряются.
- Отмена и крайний срок передаются через поток.
- Резервный маршрут проверяет требуемые возможности.
- Есть воспроизводимый эталон записи и воспроизведения.

Модуль 2: Структурированные ответы и встроенные вызовы tool

Сначала строятся независимый от поставщика протокол вызова tool и безопасный исполнитель. Затем один набор тестов поведения применяется к OpenAI Responses, механизму вызова tool Anthropic и вызову функций Gemini.

Ключевые темы

1. JSON Schema и генерация с ограничениями.
2. Сопоставление вызова с результатом.
3. Параллельные вызовы и потоковая передача.
4. Повторы, ограничение времени, отмена и идиempотентность.
5. Реестр tool и динамическое обнаружение.
6. Программные вызовы tool и отложенная загрузка.
7. Политики доступа, секреты и изолированная среда.



Набор промптов

Промпт 2.1: Протокол tool с нуля

После краткого объяснения сам реализуй ToolSpec, ToolCall, ToolResult, ToolError и ToolExecutionPolicy. Разбери обязательность полей (required), допустимость null (nullable), объединения вариантов и discriminator, границы значений, additionalProperties, версии схем и поддерживаемые поставщиками подмножества JSON Schema. Ответ LLM должен быть только предложением; выполнение начинается после проверки схемы и прав доступа.

Промпт 2.2: Реестр и исполнитель

Проведи разбор, затем расширь реализацию до асинхронных ToolRegistry и ToolExecutor. Реестр хранит неизменяемое имя, версию, хеш схемы, владельца, уровень риска, области полномочий и видимость для арендаторов. Исполнитель поддерживает крайний срок, отмену, ограниченную параллельность, предел размера результата, артефакты и сведения о ходе выполнения. Добавь детерминированные имитации tool и тесты свойств.

Промпт 2.3: Параллельное и потоковое выполнение

Создай эксперимент с отказами для потоковых аргументов: JSON невалиден до завершающего события, вызовы приходят вперемешку, одна ветвь зависает, а вторая завершается. Требуй сборку по идентификатору вызова, проверку после завершения, выполнение с учётом зависимостей, ограниченную параллельность и явное правило объединения результатов. Проверь частичный успех и отмену всего запуска.

Промпт 2.4: Повторы и побочные эффекты

Составь со мной матрицу повторов для сетевой ошибки, ограничения частоты, превышения времени, ошибки проверки, бизнес-ошибки и неизвестного исхода. Сам реализуй ключ идиempотентности, устранение дублей и сверку результата для

`send_email` и `charge_account`. Не принимай обещание выполнения «ровно один раз» без доказанной транзакционной границы.

Промпт 2.5: Адаптер OpenAI Responses

После того как собственный протокол прошёл контрактные тесты, реализуй адаптер к актуальному OpenAI Responses API: `function_call` → выполнение в приложении → `function_call_output` с тем же `call_id`, строгая схема, параллельные и потоковые вызовы. Проверь официальную документацию, зафиксируй дату/версию и прогони общий набор контрактных тестов. SDK не должен заменить наш исполнитель.

Промпт 2.6: Адаптер Anthropic

Реализуй адаптер нашего протокола к Anthropic с поддержкой `tool_use/tool_result`, `tool_use_id`, нескольких результатов в следующем сообщении пользователя, строгой схемы входа и отдельного `output_config.format`. Добавь случай детального потокового ответа с незавершёнными аргументами. Сравни формат обмена с нашим общим контрактом.

Промпт 2.7: Адаптер Gemini

Реализуй адаптер нашего протокола к Gemini с поддержкой `function_call/function_result`, `call_id`, параллельных и составных вызовов и непрозрачных подписей рассуждений. Любое сочетание вызовов `tool` со структурированным итоговым ответом пропускай через проверку возможностей. Зафиксируй предварительный или общедоступный статус функции (`preview/GA`) и добавь тест с несовместимой LLM.

Промпт 2.8: Безопасность tool

Построй модель угроз: вредоносные аргументы и результаты, обход путей, внедрение команд/SQL, SSRF, атаки через символьные ссылки, утечка секретов и подмена схемы. Сам реализуй `PolicyEngine`, проверку прав для каждого действия, службу выдачи секретов, отдельные возможности чтения и записи, список разрешённых внешних адресов и уровень ручного подтверждения. Фильтр по словам запрещён.

Промпт 2.9: Динамическое обнаружение

Добавь динамическое обнаружение без автоматического доверия. Требуй владельца, подпись и хеш схемы, неизменяемый снимок доступных `tool` на один запуск и ручное подтверждение новой или изменённой возможности. Сравни статический реестр, отложенную загрузку, поиск `tool` средствами поставщика и MCP. Измерь точность выбора `tool` и затраты токенов при 10/100/1000 схемах.

Промпт 2.10: Программные вызовы tool

Реализуй независимый от поставщика ограниченный путь оркестрации через код: LLM может написать программу, которая вызывает только разрешённые `tool` чтения, обрабатывает большие промежуточные результаты и возвращает типизированный агрегат. Раздели идентификаторы программы и вызовов `tool`, сохрани цепочку вызывающих компонентов, изолированную среду и бюджеты. Сравни прямые последовательные, параллельные и программные вызовы по качеству, подтверждающим данным, токенам, задержке и стоимости. Побочные эффекты и ручное подтверждение оставь вне программы.

Критерии прохождения модуля

- Три адаптера поставщиков проходят один набор тестов поведения.
 - Проверка схемы и прав доступа предшествует выполнению.
 - Секреты не видны LLM.
 - Параллельные вызовы сопоставляются с результатами независимо от порядка.
 - Набор тестов отказов подтверждает, что в заявленной границе операция наблюдается как выполненная один раз.
-

Модуль 3: Проектирование context

`context` — это сформированный вход одного вызова LLM. Он не равен состоянию, памяти, контрольной точке, RAG или кэшу.

Ключевые темы

1. Управление окном context, манифест и бюджеты.
2. Скользящее окно и влияние позиции.
3. Сжатие prompt и смысловое сжатие.
4. Сводки и накопление искажений.
5. Извлечение долгосрочного context.
6. Кэши KV, префикса и поставщика.
7. Происхождение данных и косвенное внедрение инструкций.



Набор промптов

Промпт 3.1: Разделение понятий

Раздели context, каноническое состояние, рабочую и долгосрочную память, контрольную точку, журнал событий, RAG и кэш. Для Agent, работающего от входного запроса до результата, перечисли источники context. Для каждого фрагмента укажи источник, уровень доверия, приоритет, свежесть, TTL, оценку числа токенов и причину включения. Сам оформи это как типизированные контракты и затем проведи со мной короткий разбор кода.

Промпт 3.2: Сборщик context

После объяснения сам реализуй `ContextSource`, `ContextCandidate`, `ContextManifest`, `ContextManager` и `PromptBuilder`. Сборщик резервирует бюджеты, ранжирует по уместности, приоритету, доверию и свежести, сжимает данные и объясняет причину включения. `PromptBuilder` принимает типизированный context, а не склеенные строки. Требуй тесты эталонных снимков.

Промпт 3.3: Окно и сводки

Расширь проект: добавь скользящее окно, извлечение структурированного состояния, обновляемую и иерархическую сводки. Незавершённые обязательства, ссылки на сущности и реестр фактов храни отдельно от повествовательного текста. Создай набор данных на 200 ходов диалога и измерь полноту воспроизведения, искажение сводки, затраты токенов и потери в середине длинного context.

Промпт 3.4: Сравнение способов сжатия

Сравни сжатие через выбор ключевых фрагментов, сжатие через prompt, смысловое сжатие и сокращение результатов tool. Требуй ссылки на источники и проверку сохранения фактов. При бюджетах 4k/16k/64k сравни полный context, скользящее окно, сводку, поиск и смешанный подход. Построй границу между качеством, стоимостью и задержкой.

Промпт 3.5: Стратегия кэширования

Проведи эксперимент по кэшированию KV, префикса и prompt у поставщика: неизменяемый префикс, изменение одного токена, аннулирование записей кэша, изоляция арендаторов и конфиденциальные данные. Требуй имитатор и измерение через настоящий API с долей попаданий в кэш, TTFT и стоимостью. Отдельно объясни редактирование context как сжатие, а не кэш.

Промпт 3.6: Извлечение долгосрочного context

Сам реализуй извлечение context из истории состояния, памяти, базы знаний и хранилища артефактов. Обязательные фильтры арендатора, ACL и доверия применяются до ранжирования по уместности. Манифест обязан объяснять каждое включение. Добавь случаи с устаревшим фактом, противоречием, отсутствующим источником и данными другого пользователя.

Промпт 3.7: Внедрение инструкций через context

Помести вредоносные инструкции в документ, результат tool и память. Сборщик context должен сохранить их как недоверенные данные, не повысить их приоритет среди инструкций и не дать им изменить политику tool. Добавь метрику успешности атаки и объясни, почему разделители, очистка и классификатор не создают достаточную границу безопасности.

Промпт 3.8: Архитектурный разбор

Проведи архитектурный разбор сборщика context по корректности, стоимости, кэшируемости, приватности и удобству эксплуатации. Назови пять решений, которые сломаются на длинных запусках или при нескольких арендаторах. Сформируй план переработки P0-P2 и ADR выбора между полным context, сводкой и извлечением данных.

Критерии прохождения модуля

- Каждый вызов LLM имеет воспроизводимый `ContextManifest`.
 - Стратегия окна и сжатия выбрана экспериментально.
 - Кэш не смешивает данные арендаторов и не называется памятью.
 - Извлечённые данные не меняют политику прав доступа.
-

Модуль 4: Извлечение данных и RAG с участием Agent

RAG — это подход, при котором LLM получает найденные данные и использует их при формировании ответа. До LlamaIndex реализуются загрузка данных, лексический, векторный и смешанный поиск, повторное ранжирование, ACL и оценка. Итоговый ответ LLM не должен скрывать слабое качество поиска.

Ключевые темы

1. Загрузка данных, версии и распространение удаления.
2. BM25, плотные векторы и понятная схема работы ANN.
3. Объединение результатов, повторное ранжирование и MMR.
4. Переформулирование и декомпозиция запроса.
5. Извлечение данных из SQL, графа и API.
6. Происхождение данных, цитаты, ACL и внедрение инструкций.



Набор промптов

Промпт 4.1: Жизненный цикл загрузки данных

Сам реализуй конвейер загрузки данных с версиями: неизменяемый идентификатор источника, исходный артефакт, структурные, смысловые и родительско-дочерние фрагменты, метаданные, устранение дублей, обновление только изменившихся данных, отметки об удалении, версию модели векторного представления, повторную индексацию и откат. Добавь тесты повторной доставки, частичного отказа и удаления из текстового, векторного слоёв и кэша.

Промпт 4.2: BM25 с нуля

Сам реализуй инвертированный индекс и BM25 без готового RAG-фреймворка. Набор данных должен включать идентификаторы, коды ошибок, имена и термины, требующие точного совпадения. Измерь Recall@k, MRR и nDCG. Затем проведи со мной разбор сложности, корректности и созданных изменений.

Промпт 4.3: Плотные векторы и ANN

Добавь адаптер модели векторного представления и точный поиск по косинусному сходству. Объясни компромиссы HNSW/IVF без переписывания промышленной библиотеки ANN. Создай случаи, где поиск по плотным векторам выигрывает и проигрывает. Зафиксируй название и версию модели векторного представления, нормализацию, размерность и стратегию повторного построения векторов.

Промпт 4.4: Смешанный поиск и повторное ранжирование

Сам реализуй объединение по обратным рангам (RRF), взвешенное объединение, интерфейс Reranker и MMR. Обязательные фильтры ACL, арендатора и времени применяются до смыслового ранжирования. Сравни BM25, поиск по плотным векторам, смешанный поиск и смешанный поиск с повторным ранжированием на одном наборе данных.

Промпт 4.5: Извлечение под управлением Agent

Добавь классификатор источников, переформулирование запроса, несколько вариантов запроса, декомпозицию, поиск родительских и дочерних фрагментов и порог для ответа «данных недостаточно». Создай неоднозначные, устаревшие, противоречивые и многошаговые случаи. Оцени траекторию поиска отдельно от итоговой генерации и докажи измеримое улучшение относительно однократного поиска.

Промпт 4.6: SQL, граф и API

Расширь KnowledgeService адаптерами SQL, графа знаний и API с актуальными данными. Для SQL требуй учётную запись только для чтения, связывание запроса со схемой, список разрешённых узлов AST, EXPLAIN, ограничение времени и числа строк. Для графа покажи разрешение сущностей и обход связей. Приведи все результаты к единому пакету подтверждающих данных.

Промпт 4.7: Цитирование и безопасность

Реализуй проверку цитат: каждое утверждение связано с фрагментом подтверждающих данных, источником и версией. Добавь вредоносные документы, защищённые ACL фрагменты и удалённую версию. Метрики: корректность цитат, подтверждённость источниками, свежесть, утечка через ACL и успешность внедрения инструкций. Объясни, почему качество поиска и генерации измеряются отдельно.

Промпт 4.8: Сопоставление с LlamaIndex

Только после выполнения критериев прохождения повтори ограниченный конвейер через актуальные средства поиска, постобработчики, RouterQueryEngine и Workflows из LlamaIndex. Не используй устаревший QueryPipeline. Сопоставь компоненты с нашим сервисом, прогони тот же набор тестов и объясни, какие детали скрывает фреймворк.

Критерии прохождения модуля

- Гибридный сервис знаний с версиями работает без готового фреймворка Agent.
 - Смешанный поиск сравнивается с простыми базовыми решениями.
 - ACL применяется до выдачи подтверждающих данных LLM.
 - Управление поиском через Agent принимается только при измеримом эффекте.
-

Модуль 5: Системы памяти

Память — это сохранённые сведения, которые Agent может использовать в текущем или будущих запусках. Она проектируется как управляемая система данных с правилами записи и чтения, ранжированием, объединением и удалением, а не как векторное хранилище со всей историей чата.

Ключевые темы

1. Рабочая, краткосрочная и долгосрочная память.
2. Семантическая, эпизодическая и процедурная память.
3. Векторный и смешанный доступ.
4. Ранжирование и временная применимость.
5. Объединение, вытеснение и забывание.
6. Очистка данных, согласие пользователя и удаление.

Набор промптов

Промпт 5.1: Устройство памяти

Сначала объясни память с изоляцией арендаторов для Agent, работающего месяцами, и обнови концепт. После к практике сам спроектируй систему из неизменяемых событий, производных записей, правил записи, извлечения и ранжирования, объединения, разрешения конфликтов, вытеснения и удаления. Полная история чата не должна автоматически становиться памятью.

Промпт 5.2: Схема записи и путь сохранения

Сам определи запись с полями `tenant`, `subject`, `type`, `content`, идентификаторами исходных событий, временем применимости, уверенностью, важностью, чувствительностью данных, версиями схемы и векторного представления, TTL и состоянием удаления. Реализуй `MemoryWriter` с проверкой согласия, классификацией, устранением дублей, разрешением сущностей, обнаружением конфликтов и карантинном отравленных данных.

Промпт 5.3: Путь чтения и ранжирование

Реализуй `MemoryRetriever` и `MemoryRanker`: обязательные фильтры ACL, лексические, векторные и структурированные кандидаты, уместность, свежесть, важность, уверенность, разнообразие и временную применимость. Результат объясняет причину включения записи. Добавь случаи с устаревшими, противоречивыми данными и данными другого арендатора.

Промпт 5.4: Объединение записей

Сам реализуй преобразования «эпизод → семантический факт», «траектории выполнения → процедурная память» и сводку более высокого уровня. Требуй связь с источниками, перенос оценки уверенности и идемпотентные задания. Измерь точность объединения, число противоречий и накопление искажений в сводках.

Промпт 5.5: Вытеснение и удаление

Сравни вытеснение по TTL, LRU, пределу объёма и полезности. Реализуй отметки об удалении, замену устаревшей записи новой, удаление, экспорт и распространение изменений в текстовый, векторный слой, кэш и резервные копии. Добавь тесты одновременного чтения и удаления, а также переноса на новую версию модели векторного представления.

Промпт 5.6: Проверка памяти атаками

Внедри вредоносную инструкцию, ложное предпочтение, отравленную процедуру, подделку происхождения данных и сохранение секрета. Построй набор данных для проверки безопасности: утечка между арендаторами, запись без полномочий, активация отравленных данных и неудачное удаление. Определи ограниченный режим без сохранения состояния.

Промпт 5.7: Оценка памяти

Спроектируй эксперимент на 100 сессиях: полнота и точность извлечения уместных записей, противоречия, временная корректность, улучшение результата основной задачи, дополнительные затраты токенов, задержка и объём хранения. Сравни работу без памяти, с недавней историей, только с векторным поиском и со смешанным поиском. Не принимай память без практического улучшения.

Промпт 5.8: Защита архитектурного решения

Проведи архитектурный разбор подсистемы памяти на уровне ведущего инженера. Разбери согласованность, изоляцию арендаторов, жизненный цикл данных, миграции данных и схем, наблюдаемость и восстановление после инцидента. Назови пять причин отключить долгосрочную память в рабочей среде и оформи ADR выбора хранилища, правил ранжирования и объединения.

Критерии прохождения модуля

- Собственные хранилища рабочей, эпизодической, семантической и процедурной памяти.
 - Запись, чтение, ранжирование, объединение и вытеснение реализованы отдельно.
 - В версионированном наборе тестов не наблюдается утечек между арендаторами.
 - Память превосходит базовое решение без неё либо отключается.
-

Модуль 6: Собственная среда выполнения Agent

Центральный модуль. Запрещены LangGraph, LangChain Agents, LlamaIndex Agents/Workflows, CrewAI, AutoGen и Microsoft Agent Framework. Разрешены адаптеры поставщиков LLM, клиенты баз данных и очередей, Pydantic, средства телеметрии и тестирования.

Ключевые темы

1. Agent, клиент LLM `ModelClient`, tool и реестр `ToolRegistry`.
2. Планирование `Planner` и выполнение `Executor`.
3. Память `Memory`, управление context `ContextManager` и сборка prompt `PromptBuilder`.
4. Шина событий `EventBus`, состояние `State` и функции объединения изменений.
5. Выполнение графа `GraphExecutor` и условия завершения.
6. Повторные попытки `RetryPolicy`, рефлексия `Reflection` и хранилище контрольных точек `CheckpointStore`.
7. Проверка политик `PolicyEngine`, учёт ресурсов `UsageLedger` и оценивание `Evaluator`.



Набор промптов

Промпт 6.1: RFC среды выполнения

Сначала объясни границы собственной асинхронной и независимой от поставщика среды выполнения Agent. После к практике сам подготовь RFC. Обязательные компоненты: `Agent`, `Tool`, `ToolRegistry`, `ModelClient`, `Planner`, `Executor`, `Memory`, `EventBus`, `ContextManager`, `PromptBuilder`, `State`, `GraphExecutor`, `RetryPolicy`, `Reflection`, `CheckpointStore`, `Evaluator`, `PolicyEngine` и `UsageLedger`. Проверь распределение ответственности, зависимости и отсутствие циклических импортов.

Промпт 6.2: Предметная модель

Сам определи `Session`, `Thread`, `Run`, `Turn`, `Goal`, `Plan`, `Step`, `Action`, `Observation`, `Artifact`, `Event` и `Checkpoint`. Отдели конфигурацию от изменяемого состояния. Все канонические сущности должны сериализоваться и иметь версию. Определи инварианты и миграции, затем проверь свойства и инварианты моделей тестами свойств.

Промпт 6.3: Модель событий

Сам реализуй типизированную шину `EventBus` и оболочку события: идентификаторы события, запуска и ветки диалога, причинную связь, корреляцию, порядковый номер в пределах области, источник, версию схемы, полезную нагрузку и уровень чувствительности. В этом модуле шина работает внутри одного процесса. Раздели предметные события, телеметрию и аудит. Проверь обработку дубликатов, нарушение порядка и сокрытие чувствительных данных.

Промпт 6.4: Состояние и функции объединения изменений

Реализуй типизированные изменения состояния, оптимистическую версию и способ объединения изменений для каждого поля: `replace`, `append`, `merge-by-ID`, `max/min` и `custom`. Попроси меня доказать ассоциативность, коммутативность и

идемпотентность там, где от них зависит параллельное слияние. Добавь тесты конфликтов, повторного воспроизведения и миграции схемы.

Промпт 6.5: Planner с нуля

Сначала объясни задачу планирования, затем сам реализуй детерминированный операторный планировщик или иерархическую сеть задач (HTN) с предусловиями, эффектами и графом зависимостей. После этого разреши LLM только предлагать типизированный Plan. Компилятор проверяет выполнимость, доступные возможности, риск, бюджеты и условие завершения. Добавь тесты недопустимого, циклического и устаревшего плана.

Промпт 6.6: Executor

Реализуй Executor, который выбирает готовые шаги, повторно проверяет предусловия и права доступа, резервирует бюджет, с ограниченным параллелизмом выполняет независимые действия, сохраняет Artifact и Observation, затем принимает одно из решений: continue/replan/reflect/wait/complete/fail. Проверь отмену и частичное завершение.

Промпт 6.7: Цикл Agent

Собери явный автомат состояний
INIT→BUILD_CONTEXT→MODEL→VALIDATE→AUTHORIZE→EXECUTE→OBSERVE. Добавь ветви REPLAN/REFLECT/WAIT/COMPLETE/FAIL. У каждого перехода должны быть условие и типизированные входы и выходы. Завершение определяется условием достижения цели, крайним сроком, бюджетами шагов, токенов и стоимости, а также обнаружением повторов — не фразой LLM.

Промпт 6.8: Повторные попытки и рефлексия

Реализуй классификацию ошибок и ограниченную политику повторов RetryPolicy. Оформи Reflection как типизированную критику: вид отказа, идентификаторы доказательств, нарушенное ограничение, предлагаемое изменение и уверенность. Рефлексия не вызывает tool и имеет отдельный бюджет. Сравни варианты без рефлексии, с одной рефлексией и с внешним проверяющим. Удали рефлексию, если она не даёт измеримого улучшения.

Промпт 6.9: Контрольная точка и повторное воспроизведение

Сам реализуй атомарную контрольную точку: версии среды выполнения и состояния, текущую позицию, множества готовых и завершённых шагов, согласования, смещение в журнале событий и расход ресурсов. Секреты и открытые соединения не сериализуются. Реализуй режимы воспроизведения live, recorded, hybrid и ответвление с новой линией происхождения. Инъектируй сбой до и после фиксации состояния.

Промпт 6.10: Интеграция и защита инженерного решения

Собери версии среды выполнения v0–v5, не переписывая компоненты. Запусти две разные прикладные задачи без изменения ядра. Защити архитектуру по корректности, стабильности API, тестируемости, точкам применения политик безопасности, стоимости и удобству эксплуатации. Предложи изменения уровней P0–P2 и сравни решение с простым циклом вызова tool.

Критерии прохождения модуля

- Все обязательные компоненты реализованы без готового каркаса для Agent.
 - Две задачи используют одно ядро.
 - Детерминированная тестовая реализация полностью заменяет LLM в тестах.
 - Падение процесса, повторное воспроизведение и бюджеты имеют наблюдаемую семантику.
 - Каждое решение в цепочке LLM → политика → tool видно в трассировке.
-

Модуль 7: Устойчивое к перезапускам выполнение графа и разбор LangGraph

Среда выполнения внутри одного процесса превращается в систему диспетчеризации и исполнителей, которая переживает сбои и перезапуски. Только после этого изучается LangGraph.

Ключевые темы

1. Направленные графы, циклы и условия завершения.
2. Разветвление и последующее объединение ветвей (`fan-out` / `fan-in`), правила объединения результатов.
3. Сохраняемая очередь готовых задач, аренда заданий и защита от устаревшего исполнителя.
4. Контрольные точки, журнал событий и повторное воспроизведение.
5. Границы внешних побочных эффектов.
6. Приостановка, участие человека (HITL) и возврат к прошлому состоянию.
7. Миграция рабочего процесса и схемы состояния.



Набор промптов

Промпт 7.1: Формальная модель выполнения

Сначала на маленьком примере объясни, почему граф с циклами не является DAG. Затем сам зафиксируй формальную модель: `State`, `Node`, `Edge`, `Guard`, `Reducer`, `Scheduler`, `ReadyQueue`, `JoinPolicy`, `Interrupt` и `RunBudget`. Покажи примеры, опровергающие корректность неассоциативной функции объединения, неограниченного цикла и недетерминированного повторного воспроизведения.

Промпт 7.2: `GraphExecutor`

Сам расширь наш `GraphExecutor`: условные рёбра, циклы, разветвление, объединение по правилам `all/any/quorum`, подграфы и детерминированный диспетчер для тестов. Компилятор должен находить недостижимые узлы, несовместимую функцию объединения и некорректные условия объединения ветвей. Добавь завершение по числу шагов, крайнему сроку, токенам и стоимости.

Промпт 7.3: Устойчивые диспетчер и исполнители

Отдели диспетчер от исполнителей, добавь сохраняемую очередь готовых задач, аренду, сигнал активности и защиту от устаревшего исполнителя. Такая защита работает, только если ресурс атомарно проверяет монотонно возрастающий маркер. Инъектируй повторную доставку, устаревшего исполнителя, потерю сигнала активности и одновременную работу двух диспетчеров. Определи владельца состояния.

Промпт 7.4: Побочные эффекты

На примерах отправки письма и платежа разметь чистое вычисление, управляемую средой выполнения повторяемую операцию и внешний побочный эффект. Спроектируй таблицы исходящих и входящих сообщений (`outbox/inbox`),

идемпотентность, устранение дубликатов, сверку и компенсацию. Инъектируй сбой после внешнего эффекта, но до контрольной точки. Определи бизнес-границу, в которой операция фактически наблюдается как однократная, и зафиксируй остаточный риск.

Промпт 7.5: Приостановка и возврат к прошлому состоянию

Реализуй типизированный договор приостановки и возобновления, срок действия согласования, гонку отмены и ответвление выполнения. Возврат к прошлому состоянию не перезаписывает историю и не повторяет внешний эффект без проверки политики. Добавь границу между воспроизведением из записи и живым выполнением, а также визуализацию происхождения ветвей.

Промпт 7.6: Миграция версий

Сам обнови граф, состояние и схему tool при незавершённых запусках. Сравни три подхода: закрепить старый код, мигрировать контрольную точку или добавить адаптер совместимости. Добавь стратегию `expand/migrate/contract`, откат и тесты несовместимой версии.

Промпт 7.7: Сопоставление с LangGraph

Теперь сопоставь нашу среду выполнения с актуальным LangGraph v1: StateGraph, функциями объединения, узлами, рёбрами и Command, хранилищем контрольных точек, Store, приостановками, подграфами, потоковой выдачей и возвратом к прошлому состоянию. Для каждого элемента укажи, что скрывает LangGraph и какие гарантии для побочных эффектов и безопасности остаются ответственностью приложения.

Промпт 7.8: Эквивалентность и выбор между собственной и готовой системой

Прогони один набор поведенческих тестов и тестов отказов на собственном GraphExecutor и LangGraph. Сравни их также с универсальной системой устойчивого выполнения класса Temporal/DBOS. Подготовь ADR по детерминизму, миграциям, эксплуатации, зависимости от поставщика и стоимости. Собственный диспетчер не должен автоматически становиться выбором для рабочей среды.

Критерии прохождения модуля

- Аварийное завершение и перезапуск процесса не теряют выполняемый запуск.
- Повторная доставка не дублирует операцию в заявленной бизнес-границе.
- Для параллельного слияния явно определены алгебраические свойства.
- Незавершённый запуск переживает миграцию схемы.
- Выбор или отказ от LangGraph обоснован в ADR.

Модуль 8: Продвинутые паттерны Agent

Все паттерны реализуются через единый интерфейс и проверяются на одном наборе данных. Сначала создаётся базовый вариант, затем паттерн реализуется с нуля, после чего разбираются отказы, соответствия готовым каркасам и результат отключения отдельных частей.

Универсальный префикс к каждому промπτу

Не начинай с пересказа статьи. Сначала попроси меня определить базовый вариант и измеримую проблему. Для паттерна раскрой модель состояний и переходов, дополнительные вызовы LLM, движение данных, условия завершения, бюджеты, способы отказа и безопасность. Дай задание на реализацию с нуля, сопоставление с готовыми каркасами и оценивание. Считай паттерн полезным только при практически значимом измеримом выигрыше.

Набор промπτов

Промпт 8.1: ReAct

Примени универсальный префикс к ReAct. Реализуй `THINK→VALIDATE→ACT→OBSERVE`, не требуя раскрытия скрытой цепочки рассуждений LLM. Добавь обнаружение повторов, очистку наблюдений от недоверенных инструкций и явное условие завершения. Сравни с базовым вариантом, где LLM напрямую предлагает вызов tool.

Промпт 8.2: Plan-and-Execute

Разбери типизированный план, предусловия, диспетчер зависимостей, проверяющий компонент, согласование и причины перестроения плана. Сравни с ReAct на примерах устаревшего плана и измерь долю допустимых планов, число перестроений и задержку критического пути.

Промпт 8.3: ReWOO

Реализуй планировщик с переменными доказательств `#E1...`, компилятор ссылок, исполнителей зависимых шагов и сборщик итогового ответа. Инъецируй сломанную ссылку и неверное наблюдение. Измерь число обращений к LLM и ускорение от параллельного выполнения.

Промпт 8.4: Self-Correction

Реализуй `GENERATE→VERIFY→DIAGNOSE→PATCH` только для задач с внешней проверкой: тестами, компилятором, схемой или набором правил. Измерь долю успеха с первой и последней попытки, регрессии, внесённые исправлением, и стоимость исправления.

Промпт 8.5: Self-Reflection

Реализуй проверяющий компонент с критериями и идентификаторами доказательств, затем решения `accept/revise/escalate`. Проверяющий не вызывает tool с побочными

эффектами. Откалибруй его относительно оценок людей и сравни с обычным дополнительным вызовом LLM для генерации ответа.

Промпт 8.6: Reflexion

Реализуй цепочку: действующий Agent → обратная связь → оценивание → запись рефлексии → проверенная память двух типов, эпизодическая и процедурная. Добавь происхождение записей, TTL и проверки на отравление памяти. Измерь кривую обучения, отрицательный перенос и накладные расходы памяти.

Промпт 8.7: Self-Discover

Создай реестр модулей способов рассуждения, типизированную структуру выбора, адаптации и композиции, затем выполни её. Записывай сделанный выбор. Сравни стабильность, долю успеха и расход токенов с одной фиксированной структурой.

Промпт 8.8: Tree of Thoughts

Реализуй генератор, функцию ценности, поиск в ширину и глубину (BFS/DFS), лучевой поиск, устранение дубликатов, ограничение глубины и бюджет. Не выполняй необратимые внешние эффекты во время поиска. Сопоставь долю успеха с числом исследованных узлов и токенов, а также с точностью оценивания.

Промпт 8.9: Graph of Thoughts

Добавь граф вариантов рассуждения с происхождением данных и операциями `generate/score/refine/aggregate`, слиянием, устранением дубликатов и бюджетом графа. Поочерёдно отключи операции, измерь их вклад и сравни соотношение качества и стоимости с Tree of Thoughts.

Промпт 8.10: Hierarchical Planning

Реализуй дерево целей, договоры для контрольных этапов, рекурсивную декомпозицию с максимальной глубиной, проверку снизу вверх и объединение результатов по пути к корню. Инъектируй ошибку верхнего уровня и измерь, какую часть плана пришлось перестроить.

Промпт 8.11: Multi-Planner

Параллельно создай разные варианты плана, приведи их к единому виду, проверь выполнимость и выбери либо объедини через арбитра. Измерь разнообразие, отставание от лучшего возможного варианта, точность арбитра и стоимость; большинство голосов не считай истиной.

Промпт 8.12: Multi-Executor

Реализуй выбор исполнителя по возможностям, пул исполнителей с ограниченным параллелизмом, аренду заданий, учёт зависимостей, владение результатами, отмену и объединение. Измерь ускорение, повторную работу, конфликты и поведение при частичных отказах.

Промпт 8.13: Deep Agents

Собери испытательную среду для длинных задач: планирование, рабочую область в файловой системе, `skills`, дочерние Agent, сжатие context, результаты,

изолированную среду, контрольные точки и согласования. Объясни, почему это композиция известных механизмов, а не новый алгоритм рассуждения. Затем сравни с актуальной реализацией Deep Agents.

Промпт 8.14: Сравнение паттернов

Запусти прямой вызов LLM, детерминированный рабочий процесс, ReAct и все паттерны на неизменяемом наборе данных. Собери долю успеха, разброс результатов, число шагов, задержку, токены, стоимость и нарушения политик. Построй границу Парето и правила выбора. Удали паттерны, которые не дают значимого эффекта.

Критерии прохождения модуля

- Для всех 13 обязательных паттернов есть небольшая реализация с нуля.
 - Для каждого зафиксированы область применения, преимущества, недостатки, отказы и соответствия готовым каркасам.
 - Одно общее сравнение исключает выбор только удобных результатов.
 - Три выбранных паттерна проходят глубокие проверки отказов и безопасности.
-

Модуль 9: Системы с несколькими Agent без театра агентов

Цель — научиться доказывать необходимость нескольких Agent, а затем строить среду их координации с явными договорами, владением данными и условиями завершения. Несколько описаний ролей в одном prompt не считаются системой с несколькими Agent.

Ключевые темы

1. Базовый вариант с одним Agent и границы декомпозиции.
2. Координатор, передача управления, маршрутизация, равноправный обмен и общая доска.
3. Оболочка сообщения, почтовый ящик, каталог Agent и обнаружение возможностей.
4. Глубина делегирования, аренда заданий, бюджеты и обнаружение завершения.
5. Общая рабочая область, владение, слияние и разрешение конфликтов.
6. Передача личности, минимальные полномочия и использование чужих полномочий не по назначению (confused deputy).
7. Оценивание системы с несколькими Agent и абляционное сравнение.



Набор промптов

Промпт 9.1: Когда несколько Agent оправданы

Дай мне пять задач, где несколько Agent выглядят убедительно, но действительно нужны лишь в части случаев. Я должен выбрать простейшую достаточную архитектуру: один вызов LLM, детерминированный рабочий процесс, один Agent с набором tool, вложенный рабочий процесс или несколько Agent. Проверь решение через необходимость изоляции, параллельного выполнения, разных прав или LLM и независимого жизненного цикла. Для каждого выбора потребуй проверяемую гипотезу и базовый вариант для сравнения.

Промпт 9.2: Протокол сообщений с нуля

Сам реализуй типизированные AgentId, MessageEnvelope, CorrelationId, CausationId, Mailbox, AgentDirectory и DeliveryReceipt. Нужны типы сообщения request/reply/event, версия схемы, крайний срок, приоритет, контекст трассировки и ключ идемпотентности. Добавь тесты дубликата, нарушения порядка, необрабатываемого сообщения и истёкшего срока. Не используй готовый каркас для нескольких Agent.

Промпт 9.3: Supervisor и делегирование ограниченных возможностей

Спроектируй координатор Supervisor, который декомпозирует цель в ограниченные задачи, выбирает исполнителя по типизированным возможностям, делегирует только необходимые tool и бюджет, проверяет договор результата и умеет отменить поддереву задач. Не передавай исполнителю весь context родителя. Проведи проверку на неограниченное делегирование, умножение повторных попыток и использование делегированных полномочий не по назначению.

Промпт 9.4: Handoff и владение диалогом

Реализуй Handoff как явную передачу владения, а не как магическую смену системного prompt. Определи предусловия, проекцию context, незавершённые

обязательства, видимый пользователю переход, откат и аудит. Смоделируй отклонённую передачу, цикл $A \rightarrow B \rightarrow A$, потерянное сообщение и отмену пользователем.

Промпт 9.5: Общая рабочая область и конкурентный доступ

Сам реализуй общую доску или рабочую область с неизменяемыми результатами, оптимистическим версионированием, арендой и политикой слияния. Два исполнителя одновременно редактируют план, и один падает после внешнего эффекта. Потребуй тесты конфликтов, решение о компенсации и доказательство отсутствия молчаливой перезаписи последним изменением.

Промпт 9.6: Завершение и алгебра бюджетов

Формализуй завершение: максимальную глубину делегирования, общий бюджет шагов и бюджет каждого Agent, общий крайний срок, бюджет токенов и стоимости, обнаружение циклов и отсутствие активной работы. Проверь взаимную блокировку, активное заикливание, осиротевшего исполнителя, лавину повторов и сбой Supervisor. Потребуй метрики полезной работы, накладных расходов координации и распределения бюджета.

Промпт 9.7: Проверка безопасности системы с несколькими Agent

Построй модель угроз для Supervisor и исполнителей: подмена личности, отмыwanie полномочий, вредоносный результат, внедрение вредоносных инструкций в prompt через почтовый ящик, утечка данных между Agent и скомпрометированный исполнитель. Сам реализуй подписанный и ограниченный токен делегирования, проверку политики при каждой передаче, происхождение данных и тест отзыва полномочий.

Промпт 9.8: Сопоставление с готовыми каркасами

После выполнения критериев прохождения сопоставь наши механизмы с LangGraph, Handoff в OpenAI Agents SDK, LlamaIndex AgentWorkflow, Microsoft Agent Framework и CrewAI. Если поставщик предлагает встроенную экспериментальную поддержку нескольких Agent, добавь её как отдельный эксперимент, а не исходный вариант для сравнения. Для каждого покажи основное состояние, гарантии доставки, границу контрольной точки, модель полномочий и условия завершения. Отдельно отметить, что скрывает готовый каркас и что остаётся ответственностью приложения.

Промпт 9.9: Абляционное сравнение нескольких Agent

Запусти неизменяемый набор данных на четырёх вариантах: прямой вызов LLM, детерминированный рабочий процесс, один Agent и несколько Agent. Сравни долю успешных задач, верхние процентиля задержки, токены, стоимость, шаги координации, нарушения политик и восстановление. Используй повторные запуски и доверительные интервалы. Вариант с несколькими Agent принимается только при измеримом выигрыше или необходимой границе безопасности либо изоляции.

Критерии прохождения модуля

- Среда выполнения поддерживает Supervisor, Handoff и ограниченный параллелизм исполнителей.
- Личность, бюджет, крайний срок и контекст трассировки сохраняются при каждой передаче.
- Сценарии дубликатов, нарушения порядка, циклов и сбоев покрыты тестами.

- Вариант с несколькими Agent либо выигрывает у базового, либо удалён из проекта.
-

Модуль 10: Инженерия протоколов — MCP, A2A и AG-UI

Цель — реализовать протоколы на уровне сообщений и только после этого подключить SDK. Эти протоколы решают разные задачи: MCP даёт Agent доступ к tool и context, A2A связывает Agent между собой, а AG-UI связывает серверную часть Agent с интерактивным пользовательским интерфейсом.

Ключевые темы

1. Обнаружение возможностей, согласование и управление версиями.
2. Возможности MCP `tools`, `resources`, `prompts`, `roots`, `sampling` и `elicitation`.
3. Транспорты MCP stdio и Streamable HTTP, ход выполнения и отмена.
4. Сущности A2A `Agent Card`, `Message`, `Task`, `Part`, `Artifact` и потоковая выдача.
5. Авторизация A2A, долгие задачи и отправляемые сервером уведомления.
6. События AG-UI, общее состояние, приостановка и переподключение.
7. Соответствие спецификации, фаззинг, совместимость и безопасность протокола.



Набор промптов

Промпт 10.1: Границы протоколов

Сравни MCP, A2A и AG-UI не по маркетингу, а по границе доверия, обнаружению возможностей, транспорту, владельцу состояния, жизненному циклу, авторизации и семантике отказов. Дай десять сценариев интеграции; я должен выбрать протокол или объяснить, почему достаточно обычного HTTP и потока событий. Исправляй попытки заменить архитектуру приложения протоколом.

Промпт 10.2: MCP stdio с нуля

По актуальной спецификации MCP сам напиши минимальные клиент и сервер поверх stdio без SDK: `initialize`, согласование возможностей, `tools/list`, `tools/call`, структурированную ошибку и завершение работы. Соблюдай правила формирования и разделения сообщений JSON-RPC из спецификации, используй эталонные примеры пакетов и тесты неизвестного метода и версии. Не добавляй сетевую авторизацию к локальному stdio.

Промпт 10.3: Возможности MCP и Streamable HTTP

Расширь реализацию `resources` и `prompts`, добавив только относящиеся к задаче возможности `roots`, `sampling` и `elicitation`. Затем добавь Streamable HTTP, правила сессии, сообщения о ходе выполнения, отмену, переподключение и авторизацию. Проведи проверку угроз: проверка заголовка `Origin`, SSRF, подмена адреса через DNS (DNS rebinding), захват сессии, использование полномочий не по назначению и запрет сквозной передачи токена.

Промпт 10.4: Ядро A2A с нуля

По актуальной спецификации A2A 1.0 реализуй получение `Agent Card`, `Message`, `Task`, `Part`, `Artifact`, `SendMessage` и путь потоковой выдачи без SDK. Зафиксируй автомат состояний `Task`, связь протокольного `contextId` с задачей и поддерживаемые

интерфейсы. Создай эталонные пакеты и тесты переходов состояния; не выдумывай адрес или метод из старого черновика спецификации.

Промпт 10.5: Долгие задачи и безопасность A2A

Добавь возобновляемую долгую задачу, состояния `input-required` и `auth-required`, отмену, переподключение и отправляемое сервером уведомление там, где они оправданы. Спроектируй авторизацию по метаданным `Agent Card`, не доверяя самой карточке. Проверь повторное использование перехваченного запроса, поддельную карточку, подмену результата, случайный дубликат запроса, истёкшие учётные данные и изоляцию арендаторов.

Промпт 10.6: Среда событий AG-UI

Реализуй минимальный событийный мост между серверной частью `Agent` и пользовательским интерфейсом: жизненный цикл запуска, текстовые события и события `tool`, снимок и изменение состояния, приостановку, согласование и переподключение. Определи порядок, устранение дубликатов, обратное давление и отмену пользователем. Зафиксируй, какие события стабильны, находятся в черновике или устарели в выбранной версии. Пользовательский интерфейс не должен напрямую получать секреты среды `tool` или становиться основным владельцем состояния выполнения.

Промпт 10.7: Сквозная совместимость

Собери демонстрацию: пользовательский интерфейс через `AG-UI` запускает локальный `Agent`, тот получает `tool` для работы с репозиторием через `MCP` и делегирует ограниченную исследовательскую задачу удалённому `Agent` через `A2A`. Передай через всю цепочку трассировку, личность, крайний срок, отмену и происхождение данных. Инъектируй сбой каждого компонента и покажи, какой слой отвечает за восстановление.

Промпт 10.8: Соответствие спецификации и миграция версий

Создай набор проверок соответствия протоколу: эталонные пакеты, проверку по схеме, повреждённые и слишком большие сообщения, неизвестные поля, переход на старую версию, превышение времени, гонку отмены, фаззинг и тесты свойств. Затем спроектируй миграцию между двумя версиями без одновременного переключения всех участников. Подключи `SDK` только после сравнения трассы его сетевого обмена с нашими эталонными пакетами.

Критерии прохождения модуля

- Есть минимальные реализации `MCP`, `A2A` и `AG-UI` без `SDK`.
- Эталонные сообщения соответствуют зафиксированным версиям спецификаций.
- Авторизация, личность, трассировка, крайний срок и отмена проходят через всю цепочку.
- Студент умеет обосновать, когда протокол вообще не нужен.

Модуль 11: Внутреннее устройство фреймворков и выбор «создавать или покупать»

Цель — не «выучить пять фреймворков», а реализовать в каждом один небольшой сценарий и понять, какие наши механизмы он заменяет, какие гарантии даёт и какие риски добавляет.

Ключевые темы

1. Отказоустойчивый граф состояния в LangGraph.
2. OpenAI Agents SDK: цикл Agent, вызовы tool, передача управления, ограничения и трассировка.
3. LlamaIndex Workflows и Agents: построение системы вокруг данных.
4. Agents, Workflows и Harness в Microsoft Agent Framework.
5. CrewAI Flows и Crews как контрольный вариант для сравнения.
6. Риски миграции, зависимость от поставщика, точки расширения и наблюдаемость.



Набор промптов

Промпт 11.1: Критерии разбора фреймворка

Составь критерии технического разбора фреймворка: каноническое состояние, планировщик, цикл Agent, сохранение состояния, прерывание, повторные попытки, побочные эффекты, потоковая выдача, context, память, схема tool, несколько Agent, трассировка, тестируемость, точки расширения, версионирование и зависимость от поставщика. Для каждого критерия потребуй исполняемый проверочный пример, а не пересказ документации.

Промпт 11.2: Разбор LangGraph

Перенеси один поток, устойчивый к падению процесса, из нашего исполнителя графа GraphExecutor в актуальный LangGraph. Сопоставь состояние, узел, ребро, команду и маршрутизацию, механизм контрольных точек, прерывание и повторное воспроизведение. Проведи тесты падения до и после побочного эффекта и сравни трассу с нашей эталонной средой выполнения. Отметь смысловые различия, которые нельзя скрыть адаптером.

Промпт 11.3: Разбор OpenAI Agents SDK

Реализуй тот же небольшой сценарий в актуальном OpenAI Agents SDK. Разбери завершение цикла Agent, вызовы tool, передачу управления, защитные ограничения, сессии, context и трассировку. Используй абстракцию поставщика там, где это действительно возможно, и явно зафиксируй оставшуюся зависимость от него. Сравни ответственность за ошибки с нашей средой выполнения.

Промпт 11.4: Разбор LlamaIndex

Реализуй вариант с большим объёмом данных через актуальные Agents и Workflows в LlamaIndex. Исследуй модель событий, состояние, абстракции поиска и данных, участие человека и совместную работу нескольких Agent. Не считай встроенную

память доказательством корректности: прогони наши тесты поиска, удаления, контрольных точек и изоляции арендаторов.

Промпт 11.5: Microsoft Agent Framework и CrewAI

Проведи контролируемое сравнение Microsoft Agent Framework и CrewAI на одном сценарии. Отдельно сравни детерминированный поток с автономной группой Agent, а Microsoft Agent Framework Harness — с нашей средой для длительных запусков. Сопоставь устойчивость к сбоям, подтверждения человеком, сжатие context, память, телеметрию и способ развёртывания. Старые руководства по AutoGen и Semantic Kernel используй только как материалы по миграции; сначала проверь актуальный статус и официальный путь миграции.

Промпт 11.6: Проверка отказов через публичный интерфейс

Для всех вариантов на фреймворках запусти одинаковые проверки: неверные аргументы tool, повтор результата, превышение времени, отмена, переполнение context, повреждение контрольной точки, падение процесса и несовместимая миграция состояния. Заполни таблицу доказательств: «заявлено в документации / подтверждено наблюдением / остаётся ответственностью приложения».

Промпт 11.7: Запись об архитектурном решении

Напиши со мной ADR о выборе между собственной реализацией и готовым решением. Варианты: собственная среда выполнения, LangGraph, OpenAI Agents SDK, LlamaIndex, Microsoft Agent Framework, CrewAI и гибриды. Оцени соответствие задаче, гарантии, экосистему, удобство эксплуатации, безопасность, переносимость, стоимость миграции и навыки команды. Выбери фреймворк только для конкретной границы эксплуатируемой системы, а не «лучший вообще».

Критерии прохождения модуля

- Один набор тестов поведения и отказов работает на всех кандидатах.
- Есть сопоставление внутренних механизмов по исходному коду и через публичный интерфейс.
- Фреймворк выбран через ADR, а не по популярности.
- Собственная среда выполнения остаётся исполняемой спецификацией и эталоном для тестов.

Модуль 12: Безопасность и Agent высокого риска

Цель — проектировать Agent как потенциально скомпрометированный компонент, принимающий решения. Ни результат LLM, ни найденные материалы, ни удалённый tool или Agent не являются доверенными инструкциями.

Ключевые темы

1. Защищаемые активы, границы доверия и характерные для Agent сценарии атак.
2. Прямое и косвенное внедрение вредоносных инструкций через запрос или данные; разделение инструкций и данных.
3. Идентификация, делегирование прав, подтверждения человеком и аудит.
4. Посредник для tool, песочница, исходящие подключения и изоляция секретов.
5. Отравление памяти и RAG, происхождение данных.
6. Безопасность цепочки поставки MCP, A2A и tool.
7. Agent для работы с кодом, браузером, графическим интерфейсом и голосом.
8. Атакующее тестирование, реагирование на инциденты и безопасная деградация.

Набор промптов

Промпт 12.1: Модель угроз как исполняемый артефакт

Построй модель угроз для Evidence & Change Agent: защищаемые активы, участники, точки входа, потоки данных, границы доверия и сценарии атак. Используй актуальные риски OWASP для Agent и LLM как проверочный список, но добавь угрозы, характерные для нашей системы. Для каждого риска укажи меры предупреждения и обнаружения, тест, телеметрию, ответственного и остаточный риск.

Промпт 12.2: Лаборатория внедрения вредоносных инструкций

Создай набор атакующих примеров с прямым и косвенным внедрением инструкций в веб-странице, задаче, метаданных PDF, комментарии к коду, результате tool и память. Реализуй типизированное происхождение инструкций, разделение context на зоны доверия, проверку разрешённых возможностей и проверку результата и побочного эффекта. Не выдавай «очистку prompt» за полноценную защиту. Измерь долю успешных атак и потерю полезности на наборе безопасных примеров.

Промпт 12.3: Сквозная идентификация и делегирование прав

Спроектируй цепочку идентификации пользователь → Agent → дочерний Agent → tool. Каждый переход получает отдельное полномочие, ограниченное получателем, сроком и задачей, без передачи исходного токена предъявителя (bearer token). Добавь точки принятия и применения правил доступа, дополнительное подтверждение для опасных действий, отзыв полномочий и аудит. Проверь злоупотребление доверенным посредником, действующим с более широкими полномочиями, повышение привилегий, повтор запроса и подделку подтверждения.

Промпт 12.4: Посредник для tool и песочница

Реализуй посредник для tool в отдельном процессе: разрешённые команды и API, типизированные аргументы, ограниченная область файловой системы, правила

исходящих сетевых подключений, квоты ресурсов, крайний срок, ограничение размера результата, отдельная выдача секретов и неизменяемый журнал аудита. Считай песочницу одним уровнем защиты, а не готовой границей безопасности. Проведи тесты выхода из песочницы, обычного обхода путей через . . / , абсолютные и кодированные пути, обхода через символические ссылки, атак DNS/SSRF и исчерпания процессов или ресурсов.

Промпт 12.5: Безопасный Agent для изменения кода

Сам построй поток изменения кода: получение недоверенного репозитория, изолированное рабочее дерево, правила для зависимостей, статический анализ и поиск секретов, тесты, артефакт, содержащий только изменения файлов, подтверждение человеком и отдельный субъект для развёртывания. Agent не имеет учётных данных эксплуатируемой среды. Смоделируй вредоносный тест, отравленный пакет, внедрение вредоносных инструкций в репозитории и разрушительную команду.

Промпт 12.6: Браузер, управление интерфейсом и голос

Сравни автоматизацию браузера через DOM/API, управление интерфейсом по снимкам экрана и голосовой Agent по наблюдаемости и рискам. Спроектируй классификатор операций, список разрешённых доменов и действий, явное подтверждение намерения, защиту от фишинга, обработку DTMF и персональных данных и проверку конечного состояния. Любые финансовые, юридические, учётные или разрушительные последствия требуют отдельного подтверждения.

Промпт 12.7: Отравление памяти и цепочки поставки

Атакуй память, векторный индекс, метаданные MCP-сервера, описание tool, Agent Card и удалённый артефакт. Реализуй подписанные сведения о происхождении, оценку доверия, карантин, проверку записи, ACL арендатора, распространение удаления и фиксацию версий зависимостей. Проверь отложенное срабатывание и отравление, сохраняющееся после сжатия и объединения данных.

Промпт 12.8: Повторяемая кампания атакующего тестирования

Спроектируй повторяемый набор атакующих тестов: классификацию атак, генератор мутаций, детерминированные исходные данные и прогоны против реальных зависимостей. Измеряй успех атаки, опасный побочный эффект, обнаружение, время до сдерживания и изоляции угрозы, ложные срабатывания и полезность. Для найденной критической уязвимости проведи учение по инциденту: отозвать доступ, изолировать систему, сохранить доказательства, уведомить ответственных, исправить и добавить регрессионный тест.

Промпт 12.9: Проверка безопасности перед выпуском

Проведи формальную проверку с решением о выпуске или запрете выпуска. Требуй схему потоков данных, реестр угроз, тесты правил доступа как кода, доказательства изоляции песочницы, подтверждение минимально необходимых прав, перечень секретов, целостность журнала аудита, наблюдение за злоупотреблениями, инструкцию по инцидентам и принятие остаточного риска. Не разрешай выпуск только потому, что LLM прошла модерацию содержимого.

Критерии прохождения модуля

- Модель угроз связана с исполняемыми тестами безопасности.
 - Любой побочный эффект проходит проверку правил, использует минимальные полномочия и записывается в журнал аудита.
 - Недоверенные материалы не расширяют возможности Agent.
 - Нет открытых рисков критического и высокого уровня без явного принятия риска.
-

Модуль 13: Оценка Agent, наблюдаемость и оптимизация

Цель — превратить «кажется, Agent стал лучше» в воспроизводимый процесс выпуска. Оцениваются конечный результат, ход выполнения, побочные эффекты, соблюдение правил, восстановление после отказов и стоимость.

Ключевые темы

1. Уровни оценки, зафиксированные наборы данных и альтернативные базовые решения.
2. Метрики результата, хода выполнения, tool, поиска, памяти и безопасности.
3. Многократные прогоны, разброс результатов и статистическая неопределённость.
4. Разметка людьми и откалиброванная оценка с помощью LLM.
5. Трассы, журналы и метрики OpenTelemetry; защита чувствительных данных.
6. Оценка в эксплуатации, канареечный выпуск, SLO и условия остановки при регрессии.
7. Ограничения стоимости и задержки, маршрутизация между LLM и кэширование.
8. Оптимизация prompt и правил без утечки тестовых данных.

Набор промптов

Промпт 13.1: Архитектура оценки

Спроектируй платформу оценки: версионированное описание задачи, неизменяемый набор данных, исходные тестовые данные, средство запуска, хранилище трасс, оценщики, метаданные эксперимента и отчёт. Раздели модульные и контрактные тесты, проверки компонентов, хода выполнения, всей системы, устойчивости к атакам и поведения в эксплуатации. Для каждого уровня зафиксируй ответственного, периодичность, стоимость и условие допуска к выпуску.

Промпт 13.2: Подготовка набора данных

Построй набор данных для Evidence & Change Agent: обычные, граничные, неоднозначные, невыполнимые, устаревшие, запрещённые правами и атакующие сценарии. Добавь уровни сложности, происхождение данных, разделение по времени, проверки загрязнения и руководство по разметке. До изменения архитектуры Agent создай базовые решения: прямой вызов LLM и детерминированный поток.

Промпт 13.3: Оценка хода выполнения

Реализуй оценщики, вычисляемые из событий: успешность задачи, корректность доказательств, точность и полноту выбора tool, недопустимый вызов, лишний шаг, восстановление, повтор побочного эффекта, нарушение правил и соблюдение бюджета. Отличай неудачное решение LLM от ошибки tool или среды выполнения. Проверь оценщики на искусственных трассах с заведомо правильным и неправильным поведением.

Промпт 13.4: Калибровка LLM как оценщика

Создай правила оценки с независимыми критериями и слепым попарным сравнением. Собери размеченный людьми набор для калибровки, измерь согласованность, склонность выбирать многословный ответ, первый ответ или определённый стиль, а также чувствительность к внедрению инструкций в оцениваемом результате.

Используй LLM как оценщика только после калибровки; разногласия и низкую уверенность направляй человеку.

Промпт 13.5: Повторные эксперименты и статистика

Для недетерминированного Agent проведи многократные прогоны с зафиксированными и версионированными настройками. Покажи распределение, доверительный интервал и размер эффекта, а не одно среднее значение. Учти парную схему эксперимента, множественные сравнения и редкие тяжёлые отказы. До принятия изменения задай порог практически значимого улучшения.

Промпт 13.6: Инструментирование OpenTelemetry

Спроектируй трассы для вызовов LLM, tool, поиска, памяти, планировщика, контрольной точки, перехода по протоколу и подтверждения человеком. Передавай контекст трассы и участка трассы через процессы-исполнители, MCP и A2A. Зафиксируй версию семантических соглашений, удаление чувствительных данных, выборку событий и пределы количества уникальных значений. По трассе должен восстанавливаться ход выполнения, но она не должна раскрывать секреты или полный чувствительный prompt.

Промпт 13.7: Оценка устойчивости к отказам и атакам

Подключи детерминированный посредник, который создаёт отказы: задержку, код 429, обрыв потока, неверную схему, устаревший результат поиска, повтор результата tool, падение процесса-исполнителя, повреждение контрольной точки и разрыв протокольного соединения. Объедини эти проверки с тестами внедрения вредоносных инструкций через prompt и повышения привилегий. Измерь управляемую деградацию, время восстановления, опасные побочные эффекты и полноту аудита.

Промпт 13.8: SLO и канареечный выпуск

Определи SLI/SLO: приближённую метрику успешности задачи, долю опасных побочных эффектов, задержку p95/p99, стоимость задачи, передачу человеку, долю задач, брошенных пользователем, и восстановление. Спроектируй теневой и канареечный запуск, переключатель функции, откат и аварийное отключение. Обратная связь из эксплуатации не должна автоматически становиться доверенной памятью или обучающей меткой.

Промпт 13.9: Стоимость и маршрутизация между LLM

Построй журнал потребления ресурсов по шагам и анализ Парето для качества, задержки и стоимости. Реализуй маршрутизацию с учётом возможностей: сначала малая LLM, затем при необходимости более сильная; добавь детерминированный короткий путь, кэширование стабильного префикса и предел параллелизма. Проверь ухудшение качества по уровням сложности и запрети резервный путь, если он теряет необходимую возможность или свойство безопасности.

Промпт 13.10: Собственный оптимизатор, затем готовый инструмент

Реализуй простой оптимизатор для работы вне эксплуатации: генератор кандидатов, разделение данных на обучающую, настроечную и тестовую части, целевую функцию с ограничениями качества, стоимости и безопасности, раннюю остановку и журнал экспериментов. Защити результат от утечки тестовых данных и переобучения. Только

затем сравни подход с актуальными DSPy, GEPa или другими инструментами оптимизации на том же протоколе и отложенном тестовом наборе.

Промпт 13.11: Решение о выпуске

Подготовь таблицу допуска к выпуску: изменения относительно базового решения, уверенность, регрессии по уровням сложности, результаты тестов безопасности и отказов, стоимость и пропускную способность, запас SLO, известные ограничения и доказательства возможности отката. Проведи со мной критическую проверку: я защищаю выпуск, ты выступаешь в ролях ведущего специалиста по оценке, безопасности и SRE. Решение должно воспроизводиться по сохранённым артефактам.

Критерии прохождения модуля

- Любое архитектурное изменение сравнивается с зафиксированным базовым решением.
- Есть откалиброванные оценщики результата, хода выполнения и безопасности.
- Трасса объясняет решение, побочные эффекты, восстановление и стоимость.
- Выпуск автоматически блокируется при регрессии или нарушении правил.

Модуль 14: Итоговый проект для эксплуатации и защита ведущего инженера

Цель — под контролем вывести в эксплуатацию ограниченную, но настоящую систему с Agent. Итоговый проект не обязан использовать все изученные приёмы: зрелость проявляется в удалении ненужной автономности.

Описание продукта

Evidence & Change Agent принимает версионированный инженерный запрос, исследует репозиторий и разрешённые актуальные источники, строит план с доказательствами, предлагает изменение кода или инструкцию по эксплуатации, выполняет только ограниченные и обратимые действия, запрашивает подтверждение перед действием с серьёзными последствиями, переживает падение процесса и оставляет полную трассу выполнения, журнал аудита и результаты оценки.

Обязательное ядро:

1. Два адаптера LLM и маршрутизация с учётом возможностей.
2. Безопасная среда выполнения tool, реестр, ограничение времени, правила повторов и песочница.
3. Сборщик context, гибридный поиск и типизированная память.
4. Планировщик, исполнитель и проверяющий; граф с сохранением состояния, контрольные точки и повторное воспроизведение.
5. Идемпотентные побочные эффекты, подтверждения и правила компенсации.
6. Модель угроз, набор атакующих тестов и делегирование прав.
7. OpenTelemetry, оценка до выпуска и в эксплуатации, условие допуска к выпуску.
8. Один ограниченный путь интеграции через MCP, A2A или AG-UI.

Несколько взаимодействующих Agent, Reflexion, ToT/GoT и другие приёмы добавляются только после эксперимента с отключением компонента, который подтвердил измеримую пользу.



Набор промптов

Промпт 14.1: RFC и границы проекта

Помоги мне написать RFC для эксплуатируемой системы: пользователь и его проблема, то, что не входит в цель, краткая модель угроз, простейшее базовое решение, обоснование использования Agent, архитектура, каноническое состояние, границы доверия, сроки хранения данных, SLO, план оценки, ввод в эксплуатацию и откат. Задавай вопросы как архитектурная комиссия уровня главного инженера и отклоняй расплывчатые требования.

Промпт 14.2: Минимальный сквозной каркас

Сам реализуй один минимальный сквозной путь от API/UI до LLM, одного tool только для чтения, артефакта с доказательствами и трассы. Без нескольких Agent и саморефлексии. Требуй герметичную локальную среду: полностью воспроизводимую и без незаявленных внешних зависимостей, детерминированный тест, манифест развёртывания, проверки работоспособности и готовности и минимальную панель наблюдения. Сначала докажи, какой компонент владеет состоянием на всём пути.

Промпт 14.3: Устойчивое выполнение и побочные эффекты

Расширь сквозной путь до графа, устойчивого к падению процесса: контрольные точки, продолжение запуска, версионированное состояние, журнал идемпотентности, таблицы исходящих и входящих сообщений (outbox / inbox), прерывание для подтверждения и отмена. Инъектируй падение до и после каждого внешнего побочного эффекта и обновление процесса посередине запуска. Я должен доказать, что каждый бизнес-эффект произойдёт не более одного раза, либо показать корректное последующее согласование состояния.

Промпт 14.4: Знания, context и память

Подключи гибридный поиск с ACL и сведениями о происхождении, сборщик context и типизированную память. Проведи тесты актуальности, удаления, изоляции арендаторов, отравления и переполнения context. Поочерёдно отключи RAG, память и сжатие и сравни результат. Объясни, какие данные являются источником истины, а какие — лишь подсказкой.

Промпт 14.5: Готовность по безопасности

Проведи полную проверку модели угроз и кампанию атакующего тестирования. Проверь внедрение вредоносных инструкций, попытки выхода из песочницы, цепочку поставки tool, делегирование прав, подделку подтверждения, утечку секретов, границы MCP/A2A и поведение скомпрометированной LLM. Все действия с серьёзными последствиями при ошибке должны запрещаться и оставлять запись аудита.

Промпт 14.6: Оценка качества и пропускной способности

Собери набор проверок до выпуска и проведи многократный эксперимент против базовых решений с прямым вызовом LLM и детерминированным потоком. Затем выполни нагрузочные, длительные и хаос-тесты: параллелизм, очереди, ограничения поставщика, деградация хранилища контрольных точек, потеря процесса-исполнителя и исчерпание бюджета. Построй модель пропускной способности и стоимость успешной задачи, а не одного запроса.

Промпт 14.7: Выбор фреймворка и протокола

Используя доказательства технического разбора, защити окончательный ADR о выборе между собственной реализацией и готовым решением. Собственная среда выполнения остаётся эталонной реализацией; эксплуатируемый поток может использовать выбранный фреймворк. Для интеграции MCP/A2A/AG-UI покажи соответствие обмена по сети спецификации, аутентификацию и ответственность за ошибки. Удали любой протокол, который не даёт реальной совместимости.

Промпт 14.8: Поэтапный ввод в эксплуатацию

Спроектируй этапы: воспроизведение сохранённых запросов вне эксплуатации, теневой запуск, внутренний режим только для чтения, канареечные арендаторы, ограниченные действия записи и постепенное расширение. Для каждого этапа задай входные и выходные метрики, ручной резервный процесс, аварийное отключение, откат и инструкцию дежурному. Запрети автоматическое расширение прав только из-за положительной продуктовой метрики.

Промпт 14.9: Учение по инциденту

Проведи учение из пяти связанных отказов: частичная недоступность поставщика LLM, отравленный документ, повтор побочного эффекта, задержка хранилища контрольных точек и скомпрометированный MCP-сервер. Я управляю инцидентом: обнаруживаю, локализирую, отзываю доступ, перевожу систему в ограниченный режим, восстанавливаю, согласую состояние и сохраняю доказательства. После составь разбор без поиска виноватых и добавь регрессионные тесты.

Промпт 14.10: Защита уровня главного инженера

Проведи 90-минутную защиту в ролях архитектуры, безопасности, SRE, прикладного ML и оценки качества, а также продукта. Проси показать исполняемые доказательства, а не слайды. Проверь допущения, скрытое состояние, смысл повторных попыток, достоверность оценщиков, стоимость и миграцию. Заверши списком блокирующих проблем, последующих действий и решением «нанять / не нанять» на позицию Senior AI Agent Engineer.

Финальные артефакты

1. RFC, схемы C4 и потоков данных, журнал ADR.
2. Собственный фреймворк с набором тестов и версионированными выпусками.
3. Реализация для эксплуатации и воспроизводимая среда.
4. Контракты LLM, tool и протоколов; эталонные тестовые данные.
5. Модель угроз, отчёт об атакующем тестировании и инструкция по инцидентам.
6. Версионированные наборы для оценки, оценщики и таблицы допуска к выпуску.
7. Трассы и панели OTel, SLO, модель пропускной способности и стоимости.
8. Демонстрация восстановления после падения, подтверждения, отката и безопасной деградации.

Критерии итоговой защиты

- Ни один критический путь не зависит от скрытого поведения фреймворка.
 - Система переживает падение процесса, повторную доставку и деградацию поставщика LLM.
 - Качество, безопасность и стоимость подтверждены зафиксированным набором данных и многократными прогонами.
 - Побочные эффекты с серьёзными последствиями ограничены правилами, идентификацией, подтверждением и аудитом.
 - Автор способен объяснить каждую границу и удалить неоправданный Agent.
-

Универсальный банк промптов для любой темы

Эти шаблоны добавляются после каждого предметного промпта. Подставляя название механизма, артефакта и набора данных; так один модуль превращается в полноценный цикл «понял → посмотрел реализацию → проверил → сломал → измерил».

Все команды на создание или изменение кода адресованы ИИ-тьютору. Студент управляет реализацией, читает её и проверяет доказательства, но не обязан писать код с пустого файла.

U.1: Создай понятную теорию

Создай или обнови конспект [MECHANISM] в Markdown/PDF. Начни с проблемы и простого примера, затем объясни ключевые понятия, поток выполнения, ограничения и типичные ошибки. В чате используй одну схему и не более одной таблицы. Не переходи к реализации до команды к практике.

U.2: Спланируй практику

Покажи цель запускаемого примера, дерево файлов, основные интерфейсы, ограничения, критерии приёмки и план из небольших шагов. Объясни, что намеренно упрощено. После плана самостоятельно переходи к реализации.

U.3: Реализуй и проверь

Самостоятельно реализуй [MECHANISM] в репозитории, добавь детерминированные тесты и запусти проверки. Не вставляй полный листинг в чат. Сообщи изменённые файлы, ключевые решения, результаты тестов и оставшиеся ограничения.

U.4: Проведи пошаговый разбор кода

Сначала покажи карту файлов и одну сквозную трассу выполнения. Затем объясняй по одному или двум компонентам за ответ: вход, состояние, важные ветки, ошибки и тесты. После каждого шага задавай один вопрос на предсказание поведения и жди моего ответа.

U.5: Сломай реализацию

Построй матрицу отказов для [MECHANISM]: неверные входные данные, превышение времени, отмена, повтор, изменение порядка, частичный ответ, падение, перезапуск, устаревшее состояние, деградация зависимости и несовпадение версий. Сам добавь детерминированные тесты, запусти их и покажи трассы двух наиболее важных отказов. Попроси меня предсказать результат до запуска одного из тестов.

U.6: Атакуй границы доверия

Составь сценарии атак для [MECHANISM]: внедрение вредоносных инструкций, отравленные данные, злоупотребление доверенным посредником, действующим с более широкими полномочиями, повышение привилегий, доступ между арендаторами, извлечение секретов и подмена аудита. Сам реализуй минимальные тесты безопасности. Для каждого риска укажи меру защиты, тест, телеметрию и остаточный риск, затем вместе со мной прочитай одно изменение кода, значимое для безопасности.

U.7: Проведи честное сравнение

Создай зафиксированный набор данных и минимум два более простых базовых решения для [MECHANISM]. Задай метрики конечного результата, хода выполнения, безопасности, задержки, числа токенов и стоимости. Сам проводи многократные прогоны, покажи неопределённость и порог практически значимого улучшения. Попроси меня решить: оставить, изменить или удалить.

U.8: Сопоставь с фреймворком

Только после выполнения критериев собственной реализации найди соответствия [MECHANISM] в [FRAMEWORK]: модули исходного кода, публичные абстракции, каноническое состояние, гарантии, точки расширения и риски, остающиеся на приложении. Сам реализуй небольшой вариант на фреймворке и прогони на нём тот же набор тестов поведения. Подтверди актуальную версию по официальным источникам.

U.9: Проверь понимание и обнови конспект

Задавай по одному вопросу на чтение кода, предсказание поведения, смысл ошибок и архитектурный компромисс. Если я ошибаюсь, покажи конкретную трассу или тест, а не давай абстрактную лекцию. После разбора обнови конспект и PDF: добавь реализованные примеры, мои выводы и ссылки на код.

U.10: Проведи итоговую проверку

Проведи проверку как скептически настроенный архитектор. Проверь, могу ли я проследить запрос, объяснить ключевой код, найти ошибку в небольшом изменении, прочитать результаты тестов и выбрать между собственной реализацией, готовым решением и удалением компонента. Определи, какие компоненты Agent можно заменить детерминированным кодом или одним вызовом LLM. Заверши критериями оценки и конкретными пробелами для повторения.

Рекомендуемый ритм одного модуля

1. **День 1:** теория, конспект и вопросы.
2. **День 2:** план и реализация тьютором с нуля.
3. **День 3:** пошаговый разбор кода и сквозных трасс выполнения.
4. **День 4:** эксперименты с контрактами, свойствами, отказами и безопасностью.
5. **День 5:** оценка на зафиксированном наборе данных и простые базовые решения.
6. **День 6:** сопоставление с фреймворком, общий набор тестов поведения и разбор исходного кода.
7. **День 7:** чтение изменения кода, ADR, обновление конспекта и устная защита.

Если критерии прохождения не выполнены, следующий модуль не начинается. Если механизм не превзошёл простое базовое решение и не создаёт необходимую границу изоляции или безопасности, он удаляется.

Критерий Senior AI Agent Engineer 2026

Трек завершён не тогда, когда просмотрены все темы, а когда ты способен:

1. Выбрать детерминированный поток вместо Agent и защитить это решение.
 2. Поставить ИИ-агенту задачу реализовать среду выполнения Agent и устойчивого исполнителя без фреймворка, проверить код и объяснить каждый критический путь.
 3. Объяснить и проверить жизненный цикл context, поиска и памяти.
 4. Ограничить идентификацию и права tool и Agent, правила доступа, песочницу и подтверждения.
 5. Доказать качество через оценку хода выполнения, наборы тестов отказов и безопасности и статистически честные базовые решения.
 6. Интегрировать MCP/A2A/AG-UI на уровне сетевого обмена и понимать, какой компонент отвечает за каждую ошибку.
 7. Выбрать фреймворк по исполняемым доказательствам и выполнить миграцию без потери корректности.
 8. Вывести ограниченную автономность в эксплуатацию с SLO, аудитом, откатом и реагированием на инциденты.
-

Реестр первичных источников

API и предварительные функции меняются быстро. Перед каждым промптом о фреймворке или поставщике необходимо проверить журнал изменений, зафиксировать версию спецификации и SDK, возможности LLM и подтвердить пример контрактным тестом.

- [OpenAI](#): вызов функций и использование tool.
- [OpenAI Agents SDK](#).
- [Anthropic](#): использование tool и детализированный поток событий tool.
- [Gemini](#): вызов функций.
- [спецификация MCP 2025-11-25](#) и [руководство по безопасности](#).
- [спецификация A2A 1.0](#).
- [документация AG-UI](#) и [жизненный цикл событий](#).
- [документация LangGraph](#).
- [Agent](#) и процессы [LlamaIndex](#).
- [Microsoft Agent Framework](#).
- [процессы CrewAI](#).
- [соглашения OpenTelemetry](#) для GenAI.
- [OWASP Top 10 для приложений с Agent, 2026](#).